

Mock Interviews

Start new sessions with /api/interviews/sessions/start.

company slug (optional)

DSA_CODING	DSA_THEORY
OS	DBMS
NETWORKS	SYSTEM_DESIGN

TOTAL QUESTIONS

5

Start Interview

Sessions

- Session #2

2/5 answered • COMPLETED
- Session #1

1/6 answered • COMPLETED

Company: General pool
Status: COMPLETED
Score: 5.00 / 10

Session Review & Evaluation

Question 1 DSA_CODING MEDIUM

Grade: 6/10

QUESTION PROMPT

You are given some courses and their prerequisites. The goal is to find a valid ordering of courses such that for every course, all its prerequisites are taken before it. For example, given courses [a, b, c] and prerequisites [(a, b), (b, c)], a valid ordering could be [c, b, a]. The problem is a variation of Course Schedule, but you need to return the ordering. For example, if we have courses [a, b, c, d] and prerequisites [(a, b), (b, c), (c, d)], the expected output could be [d, c, b, a].

YOUR ANSWER

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    int n,m; //n is no of courses, m is no of prerequisites
    cin >> n >> m;
    vector<vector<int>> > adj(n);
    vector<int> deg(n,0);
    for(int i=0;i<m;i++)
    {
        int x,y;
        cin >> x >> y;
        adj[y].push_back(x);
        deg[x]++;
    }
    queue<int> q;
    vector<int> res;
    for(int i=0;i<n;i++)
    {
        if(deg[i]==0)
        {
            res.push_back(i);
            q.push(i);
        }
    }
    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        for(int v:adj[u])
        {
            if(--deg[v]==0)
            {
                res.push_back(v);
                q.push(v);
            }
        }
    }
}
```

EVALUATION & FEEDBACK

Verdict: Needs Work
Feedback: The candidate shows some understanding of query optimization by mentioning indexing and database statistics. However, their answer lacks depth and concrete examples to demonstrate a comprehensive approach to optimizing a slow database query.

REFERENCE ANSWER

Problem Understanding

The problem requires finding a valid ordering of courses such that for every course, all its prerequisites are taken before it. We are given a list of courses and their prerequisites, represented as pairs of courses.

Key constraints and edge cases:

- The input consists of a list of courses and a list of prerequisites, where each prerequisite is a pair of courses.
- The output is a valid ordering of courses, or an indication that no such ordering exists.

- The graph may contain cycles, in which case no valid ordering exists.

Question 2 DSA_CODING MEDIUM

Grade: 4/10

QUESTION PROMPT

Given an array of integers, count the number of inversions in the array. An inversion is a pair of elements that are out of order, i.e., a larger element appears before a smaller element. For example, in the array [1, 20, 6, 4, 5], the inversions are (1, 20), (1, 6), (1, 4), (1, 5), (20, 6), (20, 4), (20, 5), (6, 4). The expected output for this testcase is 7.

YOUR ANSWER

dont know

EVALUATION & FEEDBACK

Verdict: Needs Work

Feedback: The candidate showed a lack of understanding in directly implementing a stack with a linked list. They diverted to an array-based approach without clear integration.

REFERENCE ANSWER

Problem Understanding

The problem requires counting the number of inversions in a given array of integers. An inversion is defined as a pair of elements that are out of order, i.e., a larger element appears before a smaller element.

Example: For the array [1, 20, 6, 4, 5], the inversions are (1, 20), (1, 6), (1, 4), (1, 5), (20, 6), (20, 4), (20, 5), (6, 4), resulting in a total of 7 inversions.

Input: An array of integers.

Output: The total number of inversions in the array.

Brute Force Approach

Question 3 DSA_CODING MEDIUM

Skipped

QUESTION PROMPT

Given a string and an integer k, find the length of the longest substring with at most k distinct characters. For example, for the string 'eceba' and k = 2, the expected output is 3 because 'ece' is the longest substring with at most 2 distinct characters.

REFERENCE ANSWER

Problem Understanding

The problem requires finding the length of the longest substring with at most k distinct characters in a given string. The input consists of a string s and an integer k, and the output is the length of the longest substring with at most k distinct characters.

Key constraints and edge cases:

- The string s can be empty.
- The integer k can be 0.
- The string s can have only one character.

Question 4 DSA_CODING MEDIUM

Skipped

QUESTION PROMPT

Write a program to implement a LFU (Least Frequently Used) cache. The cache should have a capacity and support get and put operations. When the cache is full, the least frequently used item should be evicted. Sample test case: Create an LFU cache with capacity 2, put(1, 1), put(2, 2), get(1), put(3, 3), get(2), Expected output: 1, -1

REFERENCE ANSWER

LFU Cache Implementation

Problem Understanding

The problem requires implementing a Least Frequently Used (LFU) cache with a given capacity. The cache should support two operations: get and put. The get operation retrieves the value associated with a given key if it exists in the cache; otherwise, it returns -1. The put operation inserts or updates the value associated with a given key. When the cache is full, the least frequently used item should be evicted.

Key constraints and edge cases:

- The cache has a limited capacity

Question 5 DSA_CODING MEDIUM

Skipped

QUESTION PROMPT

Given a string, find the length of the longest substring without repeating characters. For example, for the string 'abcabcbb', the expected output is 3 because 'abc' is the longest

substring without repeating characters. For the string 'bbbb', the expected output is 1 because 'b' is the longest substring without repeating characters.

REFERENCE ANSWER

Problem Understanding

The problem requires finding the length of the longest substring without repeating characters in a given string. The input is a string, and the output is an integer representing the length of the longest substring without repeating characters.

Key constraints and edge cases:

- The input string can be empty.
- The input string can contain only one character.
- The input string can contain repeating characters.

